

Machine Learning & Data Architecture Report

This document details the Machine Learning pipeline, data engineering, and mathematical modeling architecture for the VayuBudhi platform. This outlines the exact methodologies used to transform raw telemetry into confident, actionable enforcement intelligence.

1. Data Ingestion: How We Take the Data

Instead of relying on static datasets, we engineered a hybrid, real-time ingestion pipeline that handles both cloud-based datasets and edge IoT hardware.

- **The Live Stream (Cloud):** We treat a live, continuously updating Google Sheet as our primary telemetry database. Instead of a traditional REST API, our backend uses Python's pandas library to perform a direct HTTPS GET request to the sheet's raw CSV export URL (`/export?format=csv`). Every time the dashboard refreshes or the optimization engine runs, the backend pulls the freshest slice of data into memory. This provides us with live PM2.5, PM10, Temperature, Humidity, and Pressure readings for over 89+ stations in Delhi alone.
 - **The Edge IoT Pipeline (Hardware):** We built a dedicated `POST /api/ingest` FastAPI endpoint. This allows our custom physical ESP32 hardware sensors to push raw telemetry directly into our pipeline over WiFi, ensuring our ML models have hyperlocal ground-truth data in real-time.
-

2. Feature Engineering & Dataset Augmentation

Raw PM2.5 values are not enough to make accurate predictions. We augmented the dataset with critical meteorological and spatial features:

- **Meteorological Injection:** We incorporated Planetary Boundary Layer Height (PBLH) and Wind Speed.
 - **The Ventilation Index (VI):** We engineered a synthetic feature called the Ventilation Index, mathematically defined as $PBLH \times Wind\ Speed$. This is the most crucial predictor of atmospheric dispersion; a low VI means pollutants are trapped near the surface, acting as a massive multiplier for PM2.5 concentration.
-

3. ML Model Building: How We Calculate Results

We built two separate, highly specialized ML models and wrapped them in advanced statistical frameworks to ensure rigorous mathematical confidence.

A. The XGBoost Forecaster (Predictive AQI)

- **The Goal:** Predict the AQI 24 hours into the future to enable proactive, rather than reactive, interventions.
- **The Model:** We trained an `XGBoost Regressor` on the engineered feature space

(PM2.5, PM10, Temp, Humidity, Pressure, Wind Speed, PBLH). XGBoost was chosen for its unparalleled performance on tabular environmental data and its ability to capture non-linear interactions (like temperature inversions trapping smog).

B. The Random Forest Classifier (Source Attribution)

- **The Goal:** Identify exactly *what* is causing the pollution (Vehicles, Industry, or Dust) so authorities know *who* to penalize.
- **The Model:** We trained a Random Forest Classifier to act as a chemical signature detector. It analyzes the specific ratios of pollutants in the air. For example, high NO₂ + CO strongly correlates with vehicular traffic, while high SO₂ + PM2.5 indicates industrial emissions.

C. The Secret Weapon: Conformal Prediction (MAPIE)

- *This is the most mathematically rigorous part of the architecture.* Standard ML models just spit out a single guess (e.g., "The source is Industry").
 - We wrapped both of our models in the **MAPIE (Model Agnostic Prediction Interval Estimator)** library.
 - Instead of point estimates, our models output **Conformal Prediction Sets and Intervals**.
 - *For Forecasts:* It outputs a 90% confidence band (e.g., "AQI will be 140, but we are mathematically guaranteed it will fall between 130 and 150").
 - *For Attribution:* It outputs a probability distribution (e.g., Traffic 75%, Industry 20%).
 - **Why it matters:** Our Google OR-Tools routing algorithm will *refuse* to dispatch an enforcement inspector if the ML confidence bound drops below 80%, preventing costly false-positive enforcement.
-

4. Visualization: How We Display the ML Results

The ML models run entirely in the Python backend, but their outputs are piped directly into our Next.js + deck.gl geospatial dashboard for the end-user.

1. **The 1km IDW Grid:** Because we don't have sensors on every single street, we use Inverse-Distance Weighting (IDW) spatial interpolation. This takes our ML predictions and smooths them out across a 3D hexagonal grid covering the entire city at a 1km resolution (900m hex radius).
2. **Attribution Mapping:** The source attribution results from the Random Forest dynamically recolor the actual sensor dots on the map (Red for Traffic hotspots, Purple for Industrial hotspots, Yellow for Dust).
3. **The Deep Dive Panel:** When an official clicks on any district, the UI opens the "Deep Dive" panel. This exposes the raw ML inference engine directly to the user—rendering the exact probabilities (e.g., 60% Traffic, 40% Industry) and the statistical confidence bounds for that specific neighborhood, proving that our enforcement recommendations are backed by hard data.